

RDBMS Lab File

Submitted for the partial fulfilment for the
degree of
Bachelor of Technology
(Computer Science Engineering)



Submitted by:

Name

(Univ Rollno)

Department of Computer Science & Engineering
GURU NANAK DEV ENGINEERING COLLEGE
Ludhiana 141006

TABLE OF CONTENTS

1	Installation and configuration of PostgreSQL and Pgadmin on machine and preparing to accept connections from all IP addresses.	8
1.1	Installation and configuration of PostgreSQL 9.5	8
1.1.1	Introduction	8
1.1.2	Add the PostgreSQL repository	8
1.1.3	Update the package list	8
1.1.4	PostgreSQL installation	9
1.1.5	Connect to Postgres with the New UserConnect to Postgres with the New User	9
1.2	Installation and configuration of PgAdminIII	9
1.2.1	Connecting to database	10
1.3	Configuring PostgreSQL server 9.5 to accept from all IP addresses . .	10
2	Write the queries for Data Definition (CREATE, DROP and ALTER) and Data Manipulation Language (SELECT, INSERT, UPDATE and DELETE).	13
2.1	Create and Insert Table	13
2.2	Select Table	13
2.3	Update Table	13
2.4	Alter Table	14
2.5	Drop Table	15
2.6	Delete Table	15
3	Write SQL queries using operators (<,>=etc).	16
3.1	Comparison Operators	16
3.1.1	Equal-to Operator	16
3.1.2	Less-than Operator	16
3.1.3	Greater-than Operator	17
3.1.4	Greater-than,equal-to Operator	17
3.1.5	Less-than,equal-to Operator	17
4	Write SQL queries using SQL operators (Between, AND, OR, IN, Like, NULL).	18
4.1	The AND operator	18
4.2	The OR operator	18
4.3	The AND and OR operators together	19
4.4	The BETWEEN operator	19
4.5	The IN operator	19
4.6	The NULL operator	20
4.7	The LIKE operator	20
5	Write SQL query using character, number, date and group functions.	21
5.1	Character Functions:	21
5.1.1	CHARACTER TYPES:	21

5.1.2	UCase():	21
5.1.3	LCase():	21
5.1.4	SQL MID() Function	22
5.2	Numeric funtions :	22
5.2.1	SQL AVG() Function	22
5.2.2	SQL COUNT() Function	22
5.2.3	SQL FIRST() Function	23
5.2.4	SQL LAST() Function	23
5.2.5	SQL MAX() Function	23
5.2.6	SQL MIN() Function	23
5.2.7	SQL SUM() Function	24
5.3	GROUP Functions:	24
5.3.1	Group by statement	24
5.3.2	HAVING Clause:	25
5.4	Date/Time Functions:	25
6	Write SQL queries for Relational Algebra(UNION, INTERSECT and MINUS etc).	27
6.1	UNION Operator	27
6.2	INTERSECT Operator	28
6.3	MINUS Operator	29
7	Write SQL queries for extracting data from more than one table (Equi-Join, Non-Equi-Join , Outer Join)	30
7.1	INNER JOIN	30
7.2	LEFT OUTER JOIN	30
7.3	RIGHT OUTER JOIN	31
7.4	FULL OUTER JOIN	31
7.5	CROSS JOIN	32
8	Write SQL queries for sub queries, nested queries.	33
8.1	What is subquery	33
8.2	Rules of subqueries are:-	33
8.3	Subqueries with the SELECT Statement:-	33
8.4	Subqueries with the INSERT Statement:	34
8.5	Subqueries with the UPDATE Statement:	34
8.6	Subqueries with the DELETE Statement:	35
9	Creation, altering and dropping of tables and inserting rows into a table (use constraints while creating tables) examples using SELECT command.	37
9.1	What are Constraints	37
9.2	Types of Constraints Used:	37
9.2.1	NOT NULL Constraint:	37
9.2.2	Unique Constraint:	37
9.2.3	Primary Key Constraint	38
9.2.4	FOREIGN KEY Constraint	39
9.2.5	CHECK Constraint	39
9.2.6	EXCLUSION Constraint	40

9.2.7	Dropping Constraints	40
10	Write SQL queries for sub queries, nested queries.	42
10.1	Subquery	42
10.2	Rules of subqueries are:-	42
10.3	Subqueries with the SELECT Statement:-	42
10.4	Subqueries with the INSERT Statement:	43
10.5	Subqueries with the UPDATE Statement:	44
10.6	Subqueries with the DELETE Statement:	45
11	Creation, altering and dropping of tables and inserting rows into a table (use constraints while creating tables) examples using SELECT command.	46
11.1	What are Constraints	46
11.2	Types of Constraints Used:	46
11.2.1	NOT NULL Constraint:	46
11.2.2	Unique Constraint:	46
11.2.3	Primary Key Constraint	47
11.2.4	FOREIGN KEY Constraint	48
11.2.5	CHECK Constraint	48
11.2.6	EXCLUSION Constraint	49
11.2.7	Dropping Constraints	50

List of Figures

1.1	Adding the repository for PostgreSQL	8
1.2	Installing PostgreSQL	9
1.3	Connecting to PostgreSQL	10
1.4	PgAdmin III interface	11
2.1	Implementing INSERT query	13
2.2	Implementing SELECT Query	14
2.3	Implementing UPDATE query	14
2.4	Implementing ALTER Query	14
2.5	Implementing DROP Query	15
2.6	Implementing DELETE Query	15
3.1	Using equal-to operator	16
3.2	Using less-than operator	16
3.3	Using greater-than operator	17
3.4	Using greater-than , equal-to operator	17
3.5	Using less-than,equal-to Operator	17
4.1	Using AND Operator	18
4.2	Using OR Opertor	18
4.3	Using AND and OR Operator together	19
4.4	Using BETWEEN Operator	19
4.5	Using IN Operator	19
4.6	Using NULL Operator	20
4.7	Using LIKE Operator	20
4.8	Using LIKE Operator	20
5.1	Character types	21
5.2	Implementing using UPPERCASE Operator	21
5.3	Implementing LOWERCASE Operator	22
5.4	Implementing MID Operator	22
5.5	Implementing AVERAGE Operator	22
5.6	Implementing COUNT Operator	23
5.7	Implementing FIRST Function	23
5.8	Implementing LAST Function	23
5.9	Implementing MAX Function	24
5.10	Implementing MIN Function	24
5.11	Implementing SUM Function	24
5.12	Implementing GROUP BY Statement	25
5.13	Implementing HAVING Clause	25
5.14	Implementing DATE-FORMAT	26
6.1	Implementing UNION Operator	27
6.2	UNION ALL Operator	28
6.3	INTERSECT	28
6.4	INTERSECT Operator	29

6.5	MINUS Operator	29
7.1	Implementing INNER JOIN Query	30
7.2	Implementing LEFT OUTER JOIN Query	30
7.3	Implementing RIGHT OUTER JOIN Query	31
7.4	Implementing FULL OUTER JOIN Query	31
7.5	Implementing CROSS JOIN Query	32
8.1	Company Table	34
8.2	Subquery With Select Statement	34
8.3	Subquery With Update Statement	35
8.4	Subquery With Delete Statement	36
9.1	NOT NULL Constraint	37
9.2	Unique Constraint	38
9.3	Primary Key Constraint	38
9.4	Foreign Key Constraint	39
9.5	Check Constraint	40
9.6	Exclusion Constraint (Before Inserting Excluded Data)	40
9.7	Exclusion Constraint (After Inserting Excluded Data)	41
9.8	Drop Constraint	41
10.1	Comapany Table	43
10.2	Subquery With Select Statement	43
10.3	Subquery With Update Statement	44
10.4	Subquery With Delete Statement	45
11.1	NOT NULL Constraint	46
11.2	Unique Constraint	47
11.3	Primary Key Constraint	47
11.4	Foreign Key Constraint	48
11.5	Check Constraint	49
11.6	Exclusion Constraint (Before Inserting Excluded Data)	49
11.7	Exclusion Constraint (After Inserting Excluded Data)	50
11.8	Drop Constraint	50

Practical 1

Installation and configuration of PostgreSQL and Pgadmin on machine and preparing to accept connections from all IP addresses.

1.1 Installation and configuration of PostgreSQL 9.5

1.1.1 Introduction

Relational database management systems are a key component of many web sites and applications. They provide a structured way to store, organize, and access information.

PostgreSQL, or Postgres, is a relational database management system that provides an implementation of the SQL querying language. It is a popular choice for many small and large projects and has the advantage of being standards-compliant and having many advanced features like reliable transactions and concurrency without read locks.

1.1.2 Add the PostgreSQL repository

An additional Ubuntu repository can be added by appending a new line to the file `/etc/apt/sources.list` or by adding a new file in the directory `/etc/apt/sources.list.d/`. Second option is used here. The following command will add the new repository configuration file.

```
sudo sh -c 'echo "deb http://apt.postgresql.org/pub/repos/apt/ `lsb\`_release -cs`-pgdg main" >> /etc/apt/sources.list.d/pgdg.list'
```

Then we will import the GPG key of the repository so apt can check the validity of the packages.

```
wget -q https://www.postgresql.org/media/keys/ACCC4CF8.asc -O - | sudo apt-key add -s
```

1.1.3 Update the package list

After adding the repository, we will have to update the package list. Run the apt-get update command to load the list of available packages from postgresql.org.

```
sudo apt-get update
```



```
root@howtoforge:~# sudo sh -c 'echo "deb http://apt.postgresql.org/pub/repos/apt
/ `lsb_release -cs`-pgdg main" >> /etc/apt/sources.list.d/pgdg.list'
root@howtoforge:~#
root@howtoforge:~# wget -q https://www.postgresql.org/media/keys/ACCC4CF8.asc -O
- | sudo apt-key add -
OK
root@howtoforge:~#
```

Figure 1.1: Adding the repository for PostgreSQL


```

root@howtoforge:~#
root@howtoforge:~# sudo apt-get install postgresql postgresql-contrib
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following packages were automatically installed and are no longer required:
  linux-headers-3.13.0-57 linux-headers-3.13.0-57-generic
  linux-headers-3.13.0-61 linux-headers-3.13.0-61-generic
  linux-image-3.13.0-57-generic linux-image-3.13.0-61-generic
  linux-image-extra-3.13.0-57-generic linux-image-extra-3.13.0-61-generic
Use 'apt-get autoremove' to remove them.
The following extra packages will be installed:
  libpq5 libxslt1.1 pgdg-keyring postgresql-9.5 postgresql-client-9.5
  postgresql-client-common postgresql-common postgresql-contrib-9.5 ssl-cert
Suggested packages:
  postgresql-doc oidentd ident-server locales-all postgresql-doc-9.5
  libdbd-pg-perl openssl-blacklist
The following NEW packages will be installed:
  libpq5 libxslt1.1 pgdg-keyring postgresql postgresql-9.5
  postgresql-client-9.5 postgresql-client-common postgresql-common
  postgresql-contrib postgresql-contrib-9.5 ssl-cert
0 upgraded, 11 newly installed, 0 to remove and 61 not upgraded.
Need to get 6,250 kB of archives.
After this operation, 27.4 MB of additional disk space will be used.
Do you want to continue? [Y/n]

```

Figure 1.2: Installing PostgreSQL

1.1.4 PostgreSQL installation

The following command installs the PostgreSQL 9.5 on Ubuntu:

```
sudo apt-get install postgresql postgresql-contrib sudo apt-get update
```

```
\q
```

You should now be back in the postgres Linux command prompt.

1.1.5 Connect to Postgres with the New User

Let's assume that you have a Linux system account called test1 (you can create one by typing: `adduser test1`), and that you have created a Postgres role and database also called test1.

You can change to the Linux system account by typing:

```
sudo -i -u test1
```

You can then connect to the test1 database as the test1 Postgres role by typing:

```
psql
```

This will log in automatically assuming that all of the components have been configured.

If you want your user to connect to a different database, you can do so by specifying the database like this:

```
psql -d postgre
```

1.2 Installation and configuration of PgAdminIII

To get an idea of what PostgreSQL can do, we may start by firing up a graphical client. In a terminal type :

```
pgadmin3
```

```

root@howtoforge:~# su - postgres
postgres@howtoforge:~$ psql
psql (9.5.1)
Type "help" for help.

postgres=# \clear
postgres=# \conninfo
You are connected to database "postgres" as user "postgres" via socket in "/var/run/postgresql" at port "5432".
postgres-# █

```

Figure 1.3: Connecting to PostgreSQL

We will be presented with the pgAdmin III interface. Click on the "Add a connection to a server" button (top left). In the new dialog, enter the address, a description of the server, the default database, our username and our password.

1.2.1 Connecting to database

One more step is required in order to allow pgAdmin III to connect to the server, and that is to edit `pg_hba.conf` file and change the authentication method from `peer` to `md5`:

```
sudo nano /etc/postgresql/9.3/main/pg\_hba.conf
```

and change the line

```
# Database administrative login by Unix domain socket local all
postgres peer
```

to

```
# Database administrative login by Unix domain socket
local all postgres md5
```

Now we should reload the server configuration changes and connect pgAdmin III to our PostgreSQL database server.

```
sudo /etc/init.d/postgresql reload
```

1.3 Configuring PostgreSQL server 9.5 to accept from all IP addresses

1. Modify `pg_hba.conf` to add Client Authentication Record

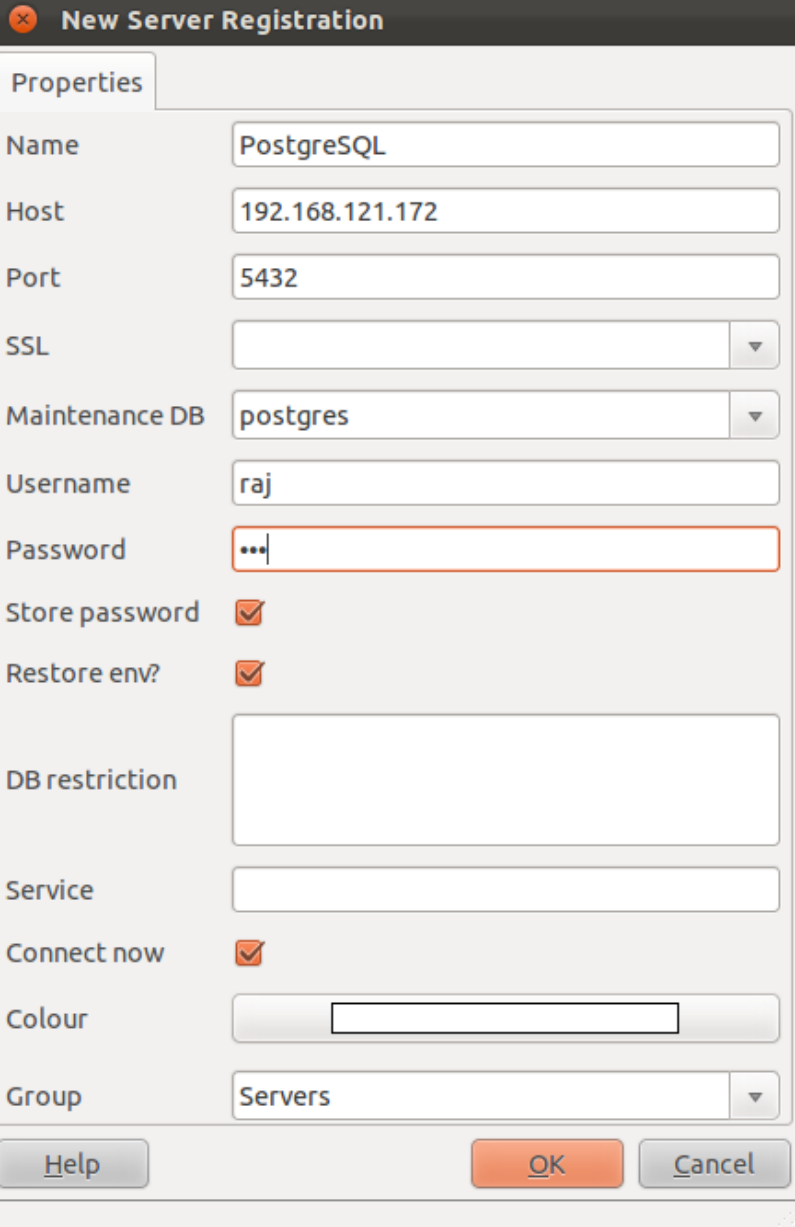
On the PostgreSQL database server, by default, we'll notice the following records towards the end of the `/var/lib/pgsql/data/pg_hba.conf`. As indicated below, it accepts connections only from the localhost.

```
# IPv4 local connections:
host all all 127.0.0.1/32 trust
# IPv6 local connections:
host all all ::1/128 ident
```

Add the following line to the `pg_hba.conf` server. This will allow connection from "192.168.101.20" ip-address.

```
# vi /var/lib/pgsql/data/pg_hba.conf
host all all 192.168.101.20/24 trust
```

2. Change the Listen Address in `postgresql.conf` On the PostgreSQL database server, by default, the listen address will be localhost in the `postgresql.conf` file as shown below.



The image shows the 'New Server Registration' dialog box in PgAdmin III. The dialog has a title bar with a close button and the text 'New Server Registration'. Below the title bar is a tab labeled 'Properties'. The main area contains several fields and checkboxes:

- Name:** PostgreSQL
- Host:** 192.168.121.172
- Port:** 5432
- SSL:** (empty dropdown menu)
- Maintenance DB:** postgres (dropdown menu)
- Username:** raj
- Password:** (password field with three dots)
- Store password:** ☒
- Restore env?:** ☒
- DB restriction:** (empty text area)
- Service:** (empty text field)
- Connect now:** ☒
- Colour:** (empty color picker)
- Group:** Servers (dropdown menu)

At the bottom of the dialog are three buttons: 'Help', 'OK', and 'Cancel'.

Figure 1.4: PgAdmin III interface

```
# grep listen /var/lib/pgsql/data/postgresql.conf
listen_addresses = 'localhost'
```

Modify this line and give *. If we have multiple interfaces on the server, we can also specify a specific interface to be listened.

```
# grep listen /var/lib/pgsql/data/postgresql.conf
listen_addresses = '*'
```

3. Test the Remote Connection Now, login to the client machine 192.168.101.20, and perform the psql remote connection to the PostgreSQL database server (192.168.102.1) as shown below. This time, it should work.

```
# psql -U postgres -h 192.168.102.1
Welcome to psql 8.1.11 (server 8.4.18), the PostgreSQL interactive
terminal
postgres=#
```

Also, if we don't want to specify the hostname in the command line parameter every time, we can setup the remote PostgreSQL database ip-address in PGHOST environment variable name as shown below.

```
# export PGHOST=192.168.102.1
# psql -U postgres
Welcome to psql 8.1.11 (server 8.4.18), the PostgreSQL interactive
terminal
postgres=#
```

Practical 2

Write the queries for Data Definition (CREATE, DROP and ALTER) and Data Manipulation Language (SELECT, INSERT, UPDATE and DELETE).

2.1 Create and Insert Table

Create is used to create a table in our database. Insert is used to insert the data in table.

```
CREATE TABLE Customer
(CustomerID int ,CustomerName varchar(255),ContactName varchar(255) ,
Address varchar(255),City varchar(255),PostalCode int ,Country
varchar(255));
```

```
INSERT INTO Customers (CustomerName, ContactName, Address, City,
PostalCode, Country)
VALUES ("Alfreds Futterkiste", "Maria Anders","Obere Str.57","Berlin
","12209","Germany");
VALUES ("Ana Trujillo Emparedados y helados", "Ana Trujillo","Avda.de
la Construction 2222","Mexico D.F. ","05021","Mexico");
VALUES ("Antonio Moreno Taqueria", "Antonio Moreno","Mataderos 2312","
Mexico D.F. ","05023","Mexico");
```

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin	12209	Germany
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.	05023	Mexico

Figure 2.1: Implementing INSERT query

2.2 Select Table

The SELECT statement is used to select data from a database.

```
SELECT * FROM Customers (limit 10);
```

2.3 Update Table

The UPDATE statement is used to update records in a table.

```
UPDATE Customers
SET ContactName="Alfred Schmidt", City="Hamburg"
WHERE CustomerName="Alfreds Futterkiste";
```

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin	12209	Germany
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.	05023	Mexico
4	Around the Horn	Thomas Hardy	120 Hanover Sq.	London	WA1 1DP	UK
5	Berglunds snabbköp	Christina Berglund	Berguvsvägen 8	Luleå	S-958 22	Sweden
6	Blauer See Delikatessen	Hanna Moos	Forsterstr. 57	Mannheim	68306	Germany
7	Blondel père et fils	Frédérique Citeaux	24, place Kléber	Strasbourg	67000	France
8	Bólido Comidas preparadas	Martin Sommer	C/ Araquil, 67	Madrid	28023	Spain
9	Bon app'	Laurence Leblhans	12, rue des Bouchers	Marseille	13008	France
10	Bottom-Dollar Marketse	Elizabeth Lincoln	23 Tsawassen Blvd.	Tsawassen	T2F 8M4	Canada

Figure 2.2: Implementing SELECT Query

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Alfred Schmidt	Obere Str. 57	Hamburg	12209	Germany
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.	05023	Mexico

Figure 2.3: Implementing UPDATE query

2.4 Alter Table

The ALTER TABLE statement is used to add, delete, or modify columns in an existing table.

```
ALTER TABLE Persons
ADD DateOfBirth date
```

P_Id	LastName	FirstName	Address	City	DateOfBirth
1	Hansen	Ola	Timoteivn 10	Sandnes	
2	Svendson	Tove	Borgvn 23	Sandnes	
3	Pettersen	Kari	Storgt 20	Stavanger	

Figure 2.4: Implementing ALTER Query

2.5 Drop Table

The DROP INDEX statement is used to delete an index in a table.

```
ALTER TABLE table_name  
DROP COLUMN DateOfBirth
```

P_Id	LastName	FirstName	Address	City
1	Hansen	Ola	Timoteivn 10	Sandnes
2	Svendson	Tove	Borgvn 23	Sandnes
3	Pettersen	Kari	Storgt 20	Stavanger

Figure 2.5: Implementing DROP Query

2.6 Delete Table

The DELETE statement is used to delete records in a table.

```
DELETE FROM Customers  
WHERE CustomerName="Alfreds Futterkiste" AND ContactName="Maria Anders"  
    ;
```

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.	05023	Mexico

Figure 2.6: Implementing DELETE Query

Practical 3

Write SQL queries using operators (<,>,=etc).

An operator is a reserved word or a character used primarily in a PostgreSQL statement's WHERE clause to perform operation(s), such as comparisons and arithmetic operations.

Operators are used to specify conditions in a PostgreSQL statement and to serve as conjunctions for multiple conditions in a statement.

3.1 Comparison Operators

It is used to compare different values.

3.1.1 Equal-to Operator

Checks if the values of two operands are equal or not, if yes then condition becomes true. (a = b) is not true.

```
SELECT * FROM Products
WHERE Price=10;
```

Number of Records: 3

ProductID	ProductName	SupplierID	CategoryID	Unit	Price
3	Aniseed Syrup	1	2	12 - 550 ml bottles	10
21	Sir Rodney's Scones	8	3	24 pkgs. x 4 pieces	10
74	Longlife Tofu	4	7	5 kg pkg.	10

Figure 3.1: Using equal-to operator

3.1.2 Less-than Operator

Checks if the value of left operand is less than the value of right operand, if yes then condition becomes true. (a < b) is true.

```
SELECT * FROM Products
WHERE Price < 10;
```

ProductID	ProductName	SupplierID	CategoryID	Unit	Price
13	Konbu	6	8	2 kg box	6
19	Teatime Chocolate Biscuits	8	3	10 boxes x 12 pieces	9.2
23	Tunnbröd	9	5	12 - 250 g pkgs.	9
24	Guaraná Fantástica	10	1	12 - 355 ml cans	4.5
33	Geitost	15	4	500 g	2.5
41	Jack's New England Clam Chowder	19	8	12 - 12 oz cans	9.65

Figure 3.2: Using less-than operator

3.1.3 Greater-than Operator

Checks if the value of left operand is greater than the value of right operand, if yes then condition becomes true.

(a > b) is not true.

```
SELECT * FROM Products
WHERE Price > 10;
```

ProductID	ProductName	SupplierID	CategoryID	Unit	Price
1	Chais	1	1	10 boxes x 20 bags	18
2	Chang	1	1	24 - 12 oz bottles	19
4	Chef Anton's Cajun Seasoning	2	2	48 - 6 oz jars	22
5	Chef Anton's Gumbo Mix	2	2	36 boxes	21.35
6	Grandma's Boysenberry Spread	3	2	12 - 8 oz jars	25
7	Uncle Bob's Organic Dried Pears	3	7	12 - 1 lb pkgs.	30

Figure 3.3: Using greater-than operator

3.1.4 Greater-than, equal-to Operator

Checks if the value of left operand is greater than or equal to the value of right operand, if yes then condition becomes true. (a >= b) is not true.

```
SELECT * FROM Products
WHERE Price >= 10;
```

ProductID	ProductName	SupplierID	CategoryID	Unit	Price
1	Chais	1	1	10 boxes x 20 bags	18
2	Chang	1	1	24 - 12 oz bottles	19
3	Aniseed Syrup	1	2	12 - 550 ml bottles	10
4	Chef Anton's Cajun Seasoning	2	2	48 - 6 oz jars	22
5	Chef Anton's Gumbo Mix	2	2	36 boxes	21.35

Figure 3.4: Using greater-than , equal-to operator

3.1.5 Less-than, equal-to Operator

Checks if the value of left operand is less than or equal to the value of right operand, if yes then condition becomes true. (a <= b) is true.

```
SELECT * FROM Products
WHERE Price <= 10;
```

ProductID	ProductName	SupplierID	CategoryID	Unit	Price
3	Aniseed Syrup	1	2	12 - 550 ml bottles	10
13	Konbu	6	8	2 kg box	6
19	Teatime Chocolate Biscuits	8	3	10 boxes x 12 pieces	9.2
21	Sir Rodney's Scones	8	3	24 pkgs. x 4 pieces	10
23	Tunnbröd	9	5	12 - 250 g pkgs.	9

Figure 3.5: Using less-than, equal-to Operator

Practical 4

Write SQL queries using SQL operators (Between, AND, OR, IN, Like, NULL).

4.1 The AND operator

The AND operator displays a record if both the first condition AND the second condition are true. The following SQL statement selects all customers from the country "UK" AND the city "London", in the "Customers" table:

```
SELECT * FROM Customers
WHERE Country="Germany"
AND City="Berlin";
```

Number of Records: 1

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin	12209	Germany

Figure 4.1: Using AND Operator

4.2 The OR operator

The OR operator displays a record if either the first condition OR the second condition is true. The following SQL statement selects all customers from the city "Berlin" OR "München", in the "Customers" table:

```
SELECT * FROM Customers
WHERE City="Berlin"
OR City="Munchen";
```

Number of Records: 2

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin	12209	Germany
25	Frankenversand	Peter Franken	Berliner Platz 43	München	80805	Germany

Figure 4.2: Using OR Operator

4.3 The AND and OR operators together

We can also combine AND and OR operators as and when required.

The following SQL statement selects all customers from the country "Germany" AND the city must be equal to "Berlin" OR "Aachen", in the "Customers" table:

```
SELECT * FROM Customers
WHERE Country="Germany"
AND ( City="Berlin" OR City="Aachen" );
```

Number of Records: 2

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin	12209	Germany
17	Drachenblut Delikatessend	Sven Ottlieb	Walsenweg 21	Aachen	52066	Germany

Figure 4.3: Using AND and OR Operator together

4.4 The BETWEEN operator

The BETWEEN operator selects values within a range. The values can be numbers, text, or dates. BETWEEN syntax is: SELECT column name(s) FROM table name WHERE column name BETWEEN value1 AND value2;

```
SELECT * FROM Products
WHERE Price BETWEEN 13.25 AND 14;
```

Number of Records: 5

ProductID	ProductName	SupplierID	CategoryID	Unit	Price
25	NuNuCa Nuß-Nougat-Creme	11	3	20 - 450 g glasses	14
34	Sasquatch Ale	16	1	24 - 12 oz bottles	14
42	Singaporean Hokkien Fried Mee	20	5	32 - 1 kg pkgs.	14
58	Escargots de Bourgogne	27	8	24 pieces	13.25
67	Laughing Lumberjack Lager	16	1	24 - 12 oz bottles	14

Figure 4.4: Using BETWEEN Operator

4.5 The IN operator

The IN operator allows us to specify multiple values in a WHERE clause.

SQL IN Syntax SELECT column name(s) FROM table name WHERE column name IN (value1,value2,...);

```
SELECT ContactName, Address, City FROM Customers
WHERE City IN ("Paris", "Berlin");
```

Number of Records: 3

ContactName	Address	City
Maria Anders	Obere Str. 57	Berlin
Marie Bertrand	265, boulevard Charonne	Paris
Dominique Perrier	25, rue Lauriston	Paris

Figure 4.5: Using IN Operator

4.6 The NULL operator

NULL values represent missing unknown data. By default, a table column can hold NULL values. If we want to select only the records with NULL values in the "Price" column, we will have to use the IS NULL operator:

```
SELECT ProductID ,ProductName ,SupplierID , CategoryID , Unit FROM Products
WHERE Price IS NULL;
```

No result.

Figure 4.6: Using NULL Operator

4.7 The LIKE operator

The LIKE operator is used to search for a specified pattern in a column. The mod sign is used to define missing letters both before and after the pattern. The following SQL statement selects all customers with a City starting with the letter "sa":

```
SELECT * FROM Customers
WHERE City LIKE "sa%";
```

Number of Records: 3

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
35	HILARIÓN-Abastos	Carlos Hernández	Carrera 22 con Ave. Carlos Soublette #8-35	San Cristóbal	5022	Venezuela
45	Let's Stop N Shop	Jaime Yorres	87 Polk St. Suite 5	San Francisco	94117	USA
59	Piccolo und mehr	Georg Pippes	Geislweg 14	Salzburg	5020	Austria

Figure 4.7: Using LIKE Operator

Also, The following SQL statement selects all customers with a Country containing the pattern "land":

```
SELECT * FROM Customers
WHERE Country LIKE "%land%";
```

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
14	Chop-suey Chinese	Yang Wang	Hauptstr. 29	Bern	3012	Switzerland
37	Hungry Owl All-Night Grocers	Patricia McKenna	8 Johnstown Road	Cork		Ireland
68	Richter Supermarkt	Michael Holz	Grenzacherweg 237	Genève	1203	Switzerland
87	Wartian Herkku	Pirkko Koskitalo	Torikatu 38	Oulu	90110	Finland

Figure 4.8: Using LIKE Operator

Practical 5

Write SQL query using character, number, date and group functions.

5.1 Character Functions:

A character set support in PostgreSQL allows you to store text in a variety of character sets including single byte character sets and multiple byte character Sets.

5.1.1 CHARACTER TYPES:

The table below lists general-purpose character types available in PostgreSQL.

Name	Description
<code>character varying(n)</code> , <code>varchar(n)</code>	variable-length with limit
<code>character(n)</code> , <code>char(n)</code>	fixed-length, blank padded
<code>text</code>	variable unlimited length

Figure 5.1: Character types

5.1.2 UCase():

The UCASE() function converts the value of a field to uppercase.

```
SELECT UCASE(CustomerName) AS Customer , City
FROM Customers;
```

Number of Records: 91	
Customer	City
ALFREDS FUTTERKISTE	Berlin
ANA TRUJILLO EMPAREADOS Y HELADOS	México D.F.
ANTONIO MORENO TAQUERÍA	México D.F.
AROUND THE HORN	London
BERGLUNDS SNABBKÖP	Luleå

Figure 5.2: Implementing using UPPERCASE Operator

5.1.3 LCase():

The LCASE() function converts the value of a field to lowercase.

```
SELECT LCASE(CustomerName) AS Customer , City
FROM Customers;
```

Number of Records: 91

Customer	City
alfreds Futterkiste	Berlin
ana trujillo emparedados y helados	México D.F.
antonio moreno taquería	México D.F.
around the horn	London

Figure 5.3: Implementing LOWERCASE Operator

5.1.4 SQL MID() Function

The MID() function is used to extract characters from a text field.

```
SELECT MID(City,1,4) AS ShortCity
FROM Customers;
```

Number of Records: 91

ShortCity
Berl
Méxi
Méxi
Lond
Lule

Figure 5.4: Implementing MID Operator

5.2 Numeric funtions :

5.2.1 SQL AVG() Function

The AVG() function returns the average value of a numeric column.

```
SELECT AVG(Price) AS PriceAverage FROM Products;
```

Number of Records: 1

PriceAverage
28.866363636363637

Figure 5.5: Implementing AVERAGE Operator

5.2.2 SQL COUNT() Function

The COUNT() function returns the number of rows that matches a specified criteria.

```
SELECT COUNT(DISTINCT CustomerID) AS NumberOfCustomers FROM Orders;
```

Number of Records: 1

NumberOfCustomers

74

Figure 5.6: Implementing COUNT Operator

5.2.3 SQL FIRST() Function

The FIRST() function returns the first value of the selected column. The FIRST() function is only supported in MS Access.

```
SELECT FIRST(CustomerName) AS FirstCustomer FROM Customers;
```

Number of Records: 1

FirstCustomer

Alfreds Futterkiste

Figure 5.7: Implementing FIRST Function

5.2.4 SQL LAST() Function

The LAST() function returns the last value of the selected column. The LAST() function is only supported in MS Access.

```
SELECT LAST(CustomerName) AS LastCustomer FROM Customers;
```

Number of Records: 1

LastCustomer

Wolski

Figure 5.8: Implementing LAST Function

5.2.5 SQL MAX() Function

The MAX() function returns the largest value of the selected column.

```
SELECT MAX(Price) AS HighestPrice FROM Products;
```

5.2.6 SQL MIN() Function

The MIN() function returns the smallest value of the selected column.

```
SELECT MIN(Price) AS SmallestOrderPrice FROM Products;
```

Number of Records: 1
LastCustomer
Wolski

Figure 5.9: Implementing MAX Function

Number of Records: 1
SmallestOrderPrice
2.5

Figure 5.10: Implementing MIN Function

5.2.7 SQL SUM() Function

The SUM() function returns the total sum of a numeric column.

```
SELECT SUM(Quantity) AS TotalItemsOrdered FROM OrderDetails;
```

Number of Records: 1
TotalItemsOrdered
12743

Figure 5.11: Implementing SUM Function

5.3 GROUP Functions:

5.3.1 Group by statement

Aggregate functions often need an added GROUP BY statement. The GROUP BY statement is used in conjunction with the aggregate functions to group the result-set by one or more columns.

```
SELECT Shippers.ShipperName,COUNT(Orders.OrderID) AS NumberOfOrders
FROM Orders
LEFT JOIN Shippers
ON Orders.ShipperID=Shippers.ShipperID
GROUP BY ShipperName;
```


Number of Records: 3

ShipperName	NumberOfOrders
Federal Shipping	68
Speedy Express	54
United Package	74

Figure 5.12: Implementing GROUP BY Statement

5.3.2 HAVING Clause:

A HAVING clause in SQL specifies that an SQL SELECT statement should only return rows where aggregate values meet the specified conditions. It was added to the SQL language because the WHERE keyword could not be used with aggregate functions

```
SELECT Employees.LastName, COUNT(Orders.OrderID) AS NumberOfOrders FROM  
    (Orders  
    INNER JOIN Employees  
    ON Orders.EmployeeID=Employees.EmployeeID)  
GROUP BY LastName  
HAVING COUNT(Orders.OrderID) > 10;
```

Number of Records: 8

LastName	NumberOfOrders
Buchanan	11
Callahan	27
Davolio	29
Fuller	20
King	14

Figure 5.13: Implementing HAVING Clause

5.4 Date/Time Functions:

MySQL comes with the following data types for storing a date or a date/time value in the database:

- DATE-format: YYYY-MM-DD
- DATETIME-format: YYYY-MM-DD HH:MI:SS
- DATETIME-format: YYYY-MM-DD HH:MI:SS
- TIMESTAMP-format: YYYY-MM-DD HH:MI:SS
- YEAR-format YYYY or YY

The following lists the most important built-in date functions in MySQL:

1. **DATE()**-Extracts the date part of a date or date/time expression
2. **EXTRACT()** -Returns a single part of a date/time

3. **DATE-ADD()**-Adds a specified time interval to a date
4. **DATE-SUB()**-Subtracts a specified time interval from a date
5. **DATE-DIFF()**-Returns the number of days between two dates
6. **DATE-FORMAT()**-Displays date/time data in different formats

```
SELECT ProductName, Price, FORMAT(Now(), 'YYYY-MM-DD') AS PerDate
FROM Products;
```

Number of Records: 77

ProductName	Price	PerDate
Chais	18	2016-10-31
Chang	19	2016-10-31
Aniseed Syrup	10	2016-10-31
Chef Anton's Cajun Seasoning	22	2016-10-31
Chef Anton's Gumbo Mix	21.35	2016-10-31
Grandma's Boysenberry Spread	25	2016-10-31
Uncle Bob's Organic Dried Pears	30	2016-10-31
Northwoods Cranberry Sauce	40	2016-10-31

Figure 5.14: Implementing DATE-FORMAT

Practical 6

Write SQL queries for Relational Algebra(UNION, INTERSECT and MINUS etc).

Relational database systems are expected to be equipped with a query language that can assist its users to query the database instances. There are two kinds of query languages: relational algebra and relational calculus.

Relational algebra is a procedural query language, which takes instances of relations as input and yields instances of relations as output. It uses operators to perform queries. An operator can be either unary or binary. They accept relations as their input and yield relations as their output. Relational algebra is performed recursively on a relation and intermediate results are also considered relations.

6.1 UNION Operator

The UNION operator is used to combine the results of two or more SELECT statements without returning any duplicate rows. Here the syntax goes like:

```
SELECT
column_1, column_2 FROM tbl_name_1
UNION
SELECT column_1, column_2 FROM tbl_name_2;
```

Example Query:

```
SELECT City FROM Customers
UNION
SELECT City FROM Suppliers
ORDER BY City;
```

Number of Records: 94

City
Aachen
Albuquerque
Anchorage
Ann Arbor
Annecy
Århus
Barcelona
Barquisimeto

Figure 6.1: Implementing UNION Operator

The following are rules applied to the queries:

- Both queries must return the same number of columns.
- The corresponding columns in the queries must have compatible data types.

UNION ALL

To get all rows that include duplicate, you use the UNION ALL operator.

```
SELECT City, Country FROM Customers WHERE Country='Germany'
UNION ALL
SELECT City, Country FROM Suppliers WHERE Country='Germany'
ORDER BY City;
```

Number of Records: 14

City	Country
Aachen	Germany
Berlin	Germany
Berlin	Germany
Brandenburg	Germany
Cunewalde	Germany
Cuxhaven	Germany
Frankfurt	Germany
Frankfurt a.M.	Germany

Figure 6.2: UNION ALL Operator

6.2 INTERSECT Operator

The PostgreSQL INTERSECT operator combines the result sets of two or more SELECT statements into a single result set. The INTERSECT operator returns all rows in the both result sets.

The final result set is represented by the yellow area where the circle A intersects with the circle B.

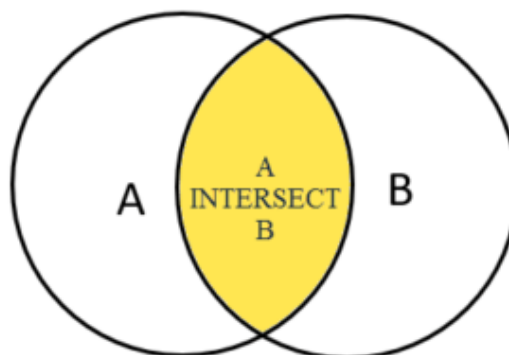


Figure 6.3: INTERSECT

Example Query:

```
SELECT ContactName FROM Customers
INTERSECT
SELECT ContactName FROM Suppliers ORDER BY ContactName;
```

ContactName
Alejandra Camino
Alexander Feuer
Ana Trujillo
Anabela Domingues
André Fonseca
Ann Devon
Anne Heikkonen
Annette Roulet

Figure 6.4: INTERSECT Operator

The following are rules applied to the queries:

- The number of columns and their order in the SELECT clauses must be the same.
- The corresponding columns in the queries must have compatible data types.

6.3 MINUS Operator

MINUS operator is used to return all rows in the first SELECT statement that are not returned by the second SELECT statement. Each SELECT statement will define a dataset. The MINUS operator will retrieve all records from the first dataset and then remove from the results all records from the second dataset.

```
SELECT ProductID FROM Orders
MINUS
SELECT ProductID FROM OrderDetails;
```

OrderDetailID	OrderID	ProductID	Quantity
1	10248	11	12
2	10248	42	10
3	10248	72	5
4	10249	14	9
5	10249	51	40
6	10250	41	10
7	10250	51	35
8	10250	65	15

Figure 6.5: MINUS Operator

Practical 7

Write SQL queries for extracting data from more than one table (Equi-Join, Non-Equi-Join , Outer Join)

JOINS are used to retrieve data from multiple tables. A JOIN is a means for combining fields from two tables by using values common to each.

7.1 INNER JOIN

INNER JOINS return all rows from multiple tables where the join condition is met.

Example:

```
SELECT Suppliersp.supplier_id , Suppliersp.supplier_name , Orders .  
       order_date  
FROM student1410891.Suppliersp  
INNER JOIN student1410891.Orders  
ON Suppliersp.supplier_id = Orders.supplier_id ;
```

supplier_id	supplier_name	order_date
10000	IBM	2013/05/12
10001	Hewlett Packard	2013/05/13

Figure 7.1: Implementing INNER JOIN Query

7.2 LEFT OUTER JOIN

This type of join returns all rows from the LEFT-hand table specified in the ON condition and only those rows from the other table where the joined fields are equal (join condition is met).

Example:

```
SELECT Suppliersp.supplier_id , Suppliersp.supplier_name , Orders .  
       order_date  
FROM student1410891.Suppliersp  
LEFT OUTER JOIN student1410891.Orders  
ON Suppliersp.supplier_id = Orders.supplier_id ;
```

supplier_id	supplier_name	order_date
10000	IBM	2013/05/12
10001	Hewlett Packard	2013/05/13
10002	Microsoft	NULL
10003	NVIDIA	NULL

Figure 7.2: Implementing LEFT OUTER JOIN Query

7.3 RIGHT OUTER JOIN

This type of join returns all rows from the RIGHT-hand table specified in the ON condition and only those rows from the other table where the joined fields are equal (join condition is met).

Example:

```
SELECT Orders.order_id , Orders.order_date , Suppliersp.supplier_name
FROM student1410891.Suppliersp
RIGHT OUTER JOIN student1410891.Orders
ON Suppliersp.supplier_id = Orders.supplier_id;
```

order_id	order_date	supplier_name
500125	2013/05/12	IBM
500126	2013/05/13	Hewlett Packard
500127	2013/05/14	NULL

Figure 7.3: Implementing RIGHT OUTER JOIN Query

7.4 FULL OUTER JOIN

This type of join returns all rows from the LEFT-hand table and RIGHT-hand table with nulls in place where the join condition is not met.

Example:

```
SELECT Suppliersp.supplier_id , Suppliersp.supplier_name , Orders.
order_date
FROM student1410891.Suppliersp
FULL OUTER JOIN student1410891.Orders
```

supplier_id	supplier_name	order_date
10000	IBM	2013/05/12
10001	Hewlett Packard	2013/05/13
NULL	NULL	2013/05/14
10002	Microsoft	NULL
10003	NVIDIA	NULL

Figure 7.4: Implementing FULL OUTER JOIN Query

7.5 CROSS JOIN

A CROSS JOIN matches every row of the first table with every row of the second table.

Example:

```
SELECT * FROM student1410891.Suppliersp  
CROSS JOIN student1410891.Orders;  
ON Suppliersp.supplier_id = Orders.supplier_id;
```

supplier_id	supplier_name	order_id	supplier_id	order_date
10000	IBM	500125	10000	2013/05/12
10001	Hewlett Packard	500125	10000	2013/05/12
10002	Microsoft	500125	10000	2013/05/12
10003	NVIDIA	500125	10000	2013/05/12
10000	IBM	500126	10001	2013/05/13
10001	Hewlett Packard	500126	10001	2013/05/13
10002	Microsoft	500126	10001	2013/05/13
10003	NVIDIA	500126	10001	2013/05/13
10000	IBM	500127	10004	2013/05/14
10001	Hewlett Packard	500127	10004	2013/05/14
10002	Microsoft	500127	10004	2013/05/14

Figure 7.5: Implementing CROSS JOIN Query

Practical 8

Write SQL queries for sub queries, nested queries.

8.1 What is subquery

- A subquery or Inner query or Nested query is a query within another PostgreSQL query and embedded within the WHERE clause.
- A subquery is used to return data that will be used in the main query as a condition to further restrict the data to be retrieved.
- Subqueries can be used with the SELECT, INSERT, UPDATE and DELETE statements along with the operators like =, <, >, >=, <=, IN, etc.

8.2 Rules of subqueries are:-

1. Subqueries must be enclosed within parentheses.
2. A subquery can have only one column in the SELECT clause, unless multiple columns are in the main query for the subquery to compare its selected columns.
3. An ORDER BY cannot be used in a subquery, although the main query can use an ORDER BY. The GROUP BY can be used to perform the same function as the ORDER BY in a subquery.
4. Subqueries that return more than one row can only be used with multiple value operators, such as the IN, EXISTS, NOT IN, ANY/SOME, ALL operator.
5. The BETWEEN operator cannot be used with a subquery; however, the BETWEEN can be used within the subquery.

8.3 Subqueries with the SELECT Statement:-

Syntax:

```
SELECT column_name [, column_name ]  
FROM   table1 [, table2 ]  
WHERE  column_name OPERATOR  
(SELECT column_name [, column_name ]  
FROM table1 [, table2 ]  
[WHERE])
```

Example:

Test Database:

```
SELECT *  
FROM COMPANY  
WHERE "ID" IN (SELECT "ID"  
FROM COMPANY  
WHERE "SALARY" > 45000) ;
```

Result:

Output pane					
	ID	NAME	AGE	ADDRESS	SALARY
	integer	text	integer	text	integer
1	1	paul	32	california	20000
2	2	allen	25	texas	15000
3	3	teddy	23	norway	20000
4	4	mark	25	rich-mond	65000
5	5	david	27	texas	85000
6	6	kim	22	south-hall	45000
7	7	james	24	houston	10000

Figure 8.1: Company Table

Output pane					
	ID	NAME	AGE	ADDRESS	SALARY
	integer	text	integer	text	integer
1	1	paul	32	california	20000
2	2	allen	25	texas	15000
3	3	teddy	23	norway	20000
4	4	mark	25	rich-mond	65000
5	5	david	27	texas	85000
6	6	kim	22	south-hall	45000
7	7	james	24	houston	10000

Figure 8.2: Subquery With Select Statement

8.4 Subqueries with the INSERT Statement:

- The INSERT statement uses the data returned from the subquery to insert into another table.
- The selected data in the subquery can be modified with any of the character, date, or number functions.

Syntax:

```
INSERT INTO table_name [ (column1 [, column2 ]) ]
SELECT [ *|column1 [, column2 ]
FROM table1 [, table2 ]
[ WHERE VALUE OPERATOR ]
```

8.5 Subqueries with the UPDATE Statement:

- The subquery can be used in conjunction with the UPDATE statement.

- Either single or multiple columns in a table can be updated when using a subquery with the UPDATE statement.

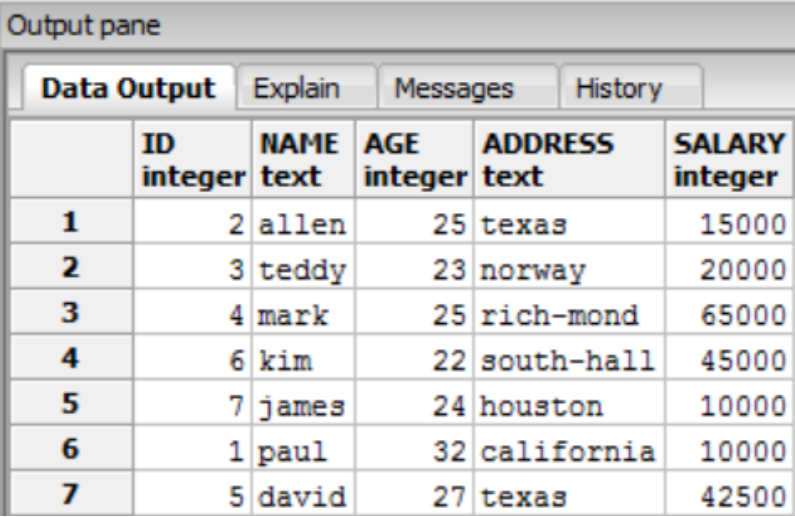
Syntax:

```
UPDATE table
SET column_name = new_value
[ WHERE OPERATOR [ VALUE ]
(SELECT COLUMN_NAME
FROM TABLE_NAME)
[ WHERE) ]
```

Example:

```
UPDATE COMPANY
SET "SALARY" = "SALARY" * 0.50
WHERE "AGE" IN (SELECT "AGE" FROM COMPANY
WHERE "AGE" >= 27 );
```

Result:



	ID integer	NAME text	AGE integer	ADDRESS text	SALARY integer
1	2	allen	25	texas	15000
2	3	teddy	23	norway	20000
3	4	mark	25	rich-mond	65000
4	6	kim	22	south-hall	45000
5	7	james	24	houston	10000
6	1	paul	32	california	10000
7	5	david	27	texas	42500

Figure 8.3: Subquery With Update Statement

8.6 Subqueries with the DELETE Statement:

The subquery can be used in conjunction with the DELETE statement like with any other statements.

Syntax:

```
DELETE FROM TABLE_NAME
[ WHERE OPERATOR [ VALUE ]
(SELECT COLUMN_NAME
FROM TABLE_NAME)
[ WHERE) ]
```

Example:

```
DELETE FROM COMPANY
WHERE "AGE" IN (SELECT "AGE" FROM COMPANY
WHERE "AGE" > 24 );
```

Result:

Output pane					
Data Output		Explain	Messages	History	
	ID integer	NAME text	AGE integer	ADDRESS text	SALARY integer
1	3	teddy	23	norway	20000
2	6	kim	22	south-hall	45000
3	7	james	24	houston	10000

Figure 8.4: Subquery With Delete Statement

Practical 9

Creation, altering and dropping of tables and inserting rows into a table (use constraints while creating tables) examples using SELECT command.

9.1 What are Constraints

- Constraints are the rules enforced on data columns on table.
- These are used to prevent invalid data from being entered into the database.
- This ensures the accuracy and reliability of the data in the database.
- Constraints could be column level or table level.
- Column level constraints are applied only to one column whereas table level constraints are applied to the whole table.

9.2 Types of Constraints Used:

9.2.1 NOT NULL Constraint:

1. Ensures that a column cannot have NULL value.
2. A NOT NULL constraint is always written as a column constraint.
3. A NULL is not the same as no data, rather, it represents unknown data.

Example:

```
CREATE TABLE COMPANY(  
ID INT PRIMARY KEY     NOT NULL,  
NAME           TEXT      NOT NULL,  
AGE            INT        NOT NULL,  
ADDRESS        CHAR(50) ,  
SALARY         REAL  
);
```

Result:

Field	Type	Null	Key	Default
ID	int(11)	NO	PRI	
NAME	text	NO		
AGE	int(11)	NO		
ADDRESS	char(50)	YES		NULL
SALARY	double	YES		NULL

Figure 9.1: NOT NULL Constraint

9.2.2 Unique Constraint:

Ensures that all values in a column are different.

Example:

```
CREATE TABLE COMPANY(
ID INT PRIMARY KEY NOT NULL,
NAME TEXT NOT NULL,
AGE INT NOT NULL UNIQUE,
ADDRESS CHAR(50),
SALARY REAL DEFAULT 50000.00
);
```

Result:

Field	Type	Null	Key	Default
ID	int(11)	NO	PRI	
NAME	text	NO		
AGE	int(11)	NO	UNI	
ADDRESS	char(50)	YES		NULL
SALARY	double	YES		50000

Figure 9.2: Unique Constraint

9.2.3 Primary Key Constraint

1. The PRIMARY KEY constraint uniquely identifies each record in a database table.
2. There can be more UNIQUE columns, but only one primary key in a table. Primary keys are important when designing the database tables. Primary keys are unique ids.
3. A table can have only one primary key, which may consist of single or multiple fields. When multiple fields are used as a primary key, they are called a composite key.
4. If a table has a primary key defined on any field(s), then you cannot have two records having the same value of that field(s).

Example:

```
CREATE TABLE COMPANY(
ID INT PRIMARY KEY NOT NULL,
NAME TEXT NOT NULL,
AGE INT NOT NULL,
ADDRESS CHAR(50),
SALARY REAL
);
```

Result:

Field	Type	Null	Key	Default
ID	int(11)	NO	PRI	
NAME	text	NO		
AGE	int(11)	NO		
ADDRESS	char(50)	YES		NULL
SALARY	double	YES		NULL

Figure 9.3: Primary Key Constraint

9.2.4 FOREIGN KEY Constraint

1. A foreign key constraint specifies that the values in a column (or a group of columns) must match the values appearing in some row of another table.
2. They are called foreign keys because the constraints are foreign; that is, outside the table.
3. Foreign keys are sometimes called a referencing key.

Example:

Creates a table called COMPANY.

```
CREATE TABLE COMPANY(  
ID INT PRIMARY KEY      NOT NULL,  
NAME                    TEXT    NOT NULL,  
AGE                     INT     NOT NULL,  
ADDRESS                 CHAR(50) ,  
SALARY                  REAL  
);
```

Creates a table called DEPARTMENT.

```
CREATE TABLE DEPARTMENT(  
ID INT PRIMARY KEY      NOT NULL,  
DEPT CHAR(50) NOT NULL,  
EMP_ID INT              references COMPANY(ID)  
);
```

Result:

Field	Type	Null	Key	Default
ID	int(11)	NO	PRI	
DEPT	char(50)	NO		
EMP_ID	int(11)	YES		NULL

Figure 9.4: Foreign Key Constraint

The column EMP_ID is the foreign key and references the ID field of the table COMPANY.

9.2.5 CHECK Constraint

1. The CHECK constraint ensures that all values in a column satisfy certain conditions.
2. If the condition evaluates to false, the record violates the constraint and isn't entered into the table.

Example:

```
CREATE TABLE COMPANY(  
ID INT PRIMARY KEY      NOT NULL,  
NAME                    TEXT    NOT NULL,  
AGE                     INT     NOT NULL,  
ADDRESS                 CHAR(50) ,  
SALARY                  REAL    CHECK(SALARY > 0)  
);
```

Result:

Field	Type	Null	Key	Default
ID	int(11)	NO	PRI	
NAME	text	NO		
AGE	int(11)	NO		
ADDRESS	char(50)	YES		NULL
SALARY	double	YES		NULL

Figure 9.5: Check Constraint

9.2.6 EXCLUSION Constraint

1. The EXCLUDE constraint ensures that if any two rows are compared on the specified column(s) or expression(s) using the specified operator(s), not all of these comparisons will return TRUE.
2. At least one of these operator comparisons will return false or null.

Example:

```
CREATE TABLE COMPANY(
ID INT PRIMARY KEY     NOT NULL,
NAME           TEXT    ,
AGE           INT      ,
ADDRESS       CHAR(50) ,
SALARY        REAL,
EXCLUDE USING gist
(NAME WITH =,
AGE WITH =)
);
```

Following data is inserted in the company table:-

```
INSERT INTO COMPANY10 VALUES(1, 'A', 20, 'abc ', 20000.00 );
INSERT INTO COMPANY10 VALUES(2, 'B', 18, 'xyz ', 25000.00 );
```

Result:

	id integer	name text	age integer	address character(50)	salary real
1	1	A	20	abc	20000
2	2	B	18	xyz	25000

Figure 9.6: Exclusion Constraint (Before Inserting Excluded Data)

On adding the third row as:

```
INSERT INTO COMPANY10 VALUES(3, 'A', 20, 'pqr ', 30000.00 );
```

The output is shown in Figure 11.7. An error is generated because the person with ID = 3 has same name and age as of person with ID = 1. Entries with same name and age cannot be entered into the table as they are excluded by using Exclusion Constraint.

Result:

9.2.7 Dropping Constraints

1. To remove a constraint you need to know its name.
2. If the name is known, it's easy to drop.

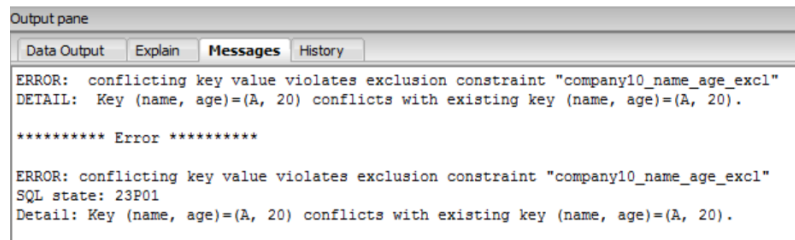


Figure 9.7: Exclusion Constraint (After Inserting Excluded Data)

3. Else you need to find out the system generated name.

Syntax:

```
ALTER TABLE table_name DROP CONSTRAINT some_name;
```

Example:

```
ALTER TABLE company MODIFY ID INT NOT NULL;  
ALTER TABLE company DROP PRIMARY KEY;
```

The above query is applied on table company Figure 11.1 and primary key of ID is dropped. Output is shown in Figure 11.8.

Result:

Field	Type	Null	Key	Default
id	int(11)	NO		
NAME	text	NO		
AGE	int(11)	NO		
ADDRESS	char(50)	YES		NULL
SALARY	double	YES		NULL

Figure 9.8: Drop Constraint

Practical 10

Write SQL queries for sub queries, nested queries.

10.1 Subquery

- A subquery or Inner query or Nested query is a query within another PostgreSQL query and embedded within the WHERE clause.
- A subquery is used to return data that will be used in the main query as a condition to further restrict the data to be retrieved.
- Subqueries can be used with the SELECT, INSERT, UPDATE and DELETE statements along with the operators like =, <, >, >=, <=, IN, etc.

10.2 Rules of subqueries are:-

1. Subqueries must be enclosed within parentheses.
2. A subquery can have only one column in the SELECT clause, unless multiple columns are in the main query for the subquery to compare its selected columns.
3. An ORDER BY cannot be used in a subquery, although the main query can use an ORDER BY. The GROUP BY can be used to perform the same function as the ORDER BY in a subquery.
4. Subqueries that return more than one row can only be used with multiple value operators, such as the IN, EXISTS, NOT IN, ANY/SOME, ALL operator.
5. The BETWEEN operator cannot be used with a subquery; however, the BETWEEN can be used within the subquery.

10.3 Subqueries with the SELECT Statement:-

Syntax:

```
SELECT column_name [, column_name ]
FROM   table1 [, table2 ]
WHERE  column_name OPERATOR
      (SELECT column_name [, column_name ]
FROM   table1 [, table2 ]
[WHERE])
```

Example:

Test Database:

```
SELECT *
FROM COMPANY
WHERE "ID" IN (SELECT "ID"
FROM COMPANY
WHERE "SALARY" > 45000) ;
```

Result:

Output pane					
	ID	NAME	AGE	ADDRESS	SALARY
	integer	text	integer	text	integer
1	1	paul	32	california	20000
2	2	allen	25	texas	15000
3	3	teddy	23	norway	20000
4	4	mark	25	rich-mond	65000
5	5	david	27	texas	85000
6	6	kim	22	south-hall	45000
7	7	james	24	houston	10000

Figure 10.1: Comapany Table

Output pane					
	ID	NAME	AGE	ADDRESS	SALARY
	integer	text	integer	text	integer
1	1	paul	32	california	20000
2	2	allen	25	texas	15000
3	3	teddy	23	norway	20000
4	4	mark	25	rich-mond	65000
5	5	david	27	texas	85000
6	6	kim	22	south-hall	45000
7	7	james	24	houston	10000

Figure 10.2: Subquary With Select Statement

10.4 Subqueries with the INSERT Statement:

- The INSERT statement uses the data returned from the subquery to insert into another table.
- The selected data in the subquery can be modified with any of the character, date, or number functions.

Syntax:

```
INSERT INTO table_name [ (column1 [, column2 ]) ]
SELECT [ *|column1 [, column2 ]
FROM table1 [, table2 ]
[ WHERE VALUE OPERATOR ]
```

10.5 Subqueries with the UPDATE Statement:

- The subquery can be used in conjunction with the UPDATE statement.
- Either single or multiple columns in a table can be updated when using a subquery with the UPDATE statement.

Syntax:

```
UPDATE table
SET column_name = new_value
[ WHERE OPERATOR [ VALUE ]
(SELECT COLUMN_NAME
FROM TABLE_NAME)
[ WHERE) ]
```

Example:

```
UPDATE COMPANY
SET "SALARY" = "SALARY" * 0.50
WHERE "AGE" IN (SELECT "AGE" FROM COMPANY
WHERE "AGE" >= 27 );
```

Result:

Output pane					
Data Output Explain Messages History					
	ID integer	NAME text	AGE integer	ADDRESS text	SALARY integer
1	2	allen	25	texas	15000
2	3	teddy	23	norway	20000
3	4	mark	25	rich-mond	65000
4	6	kim	22	south-hall	45000
5	7	james	24	houston	10000
6	1	paul	32	california	10000
7	5	david	27	texas	42500

Figure 10.3: Subquery With Update Statement

10.6 Subqueries with the DELETE Statement:

The subquery can be used in conjunction with the DELETE statement like with any other statements.

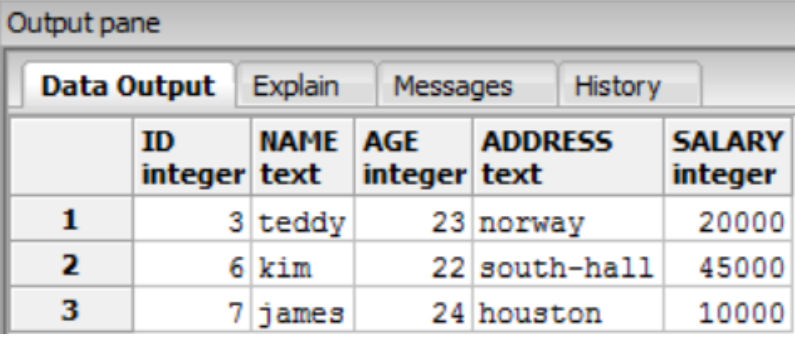
Syntax:

```
DELETE FROM TABLE_NAME  
[ WHERE OPERATOR [ VALUE ]  
(SELECT COLUMN_NAME  
FROM TABLE_NAME)  
[ WHERE ) ]
```

Example:

```
DELETE FROM COMPANY  
WHERE "AGE" IN (SELECT "AGE" FROM COMPANY  
WHERE "AGE" > 24 );
```

Result:



	ID integer	NAME text	AGE integer	ADDRESS text	SALARY integer
1	3	teddy	23	norway	20000
2	6	kim	22	south-hall	45000
3	7	james	24	houston	10000

Figure 10.4: Subquery With Delete Statement

Practical 11

Creation, altering and dropping of tables and inserting rows into a table (use constraints while creating tables) examples using SELECT command.

11.1 Constraints

- Constraints are the rules enforced on data columns on table.
- These are used to prevent invalid data from being entered into the database.
- This ensures the accuracy and reliability of the data in the database.
- Constraints could be column level or table level.
- Column level constraints are applied only to one column where as table level constraints are applied to the whole table.

11.2 Types of Constraints Used:

11.2.1 NOT NULL Constraint:

1. Ensures that a column cannot have NULL value.
2. A NOT NULL constraint is always written as a column constraint.
3. A NULL is not the same as no data, rather, it represents unknown data.

Example:

```
CREATE TABLE COMPANY(  
  ID INT PRIMARY KEY     NOT NULL,  
  NAME           TEXT     NOT NULL,  
  AGE            INT       NOT NULL,  
  ADDRESS        CHAR(50) ,  
  SALARY         REAL  
);
```

Result:

Field	Type	Null	Key	Default
ID	int(11)	NO	PRI	
NAME	text	NO		
AGE	int(11)	NO		
ADDRESS	char(50)	YES		
SALARY	double	YES		

Figure 11.1: NOT NULL Constraint

11.2.2 Unique Constraint:

Ensures that all values in a column are different.

Example:

```
CREATE TABLE COMPANY(  
  ID INT PRIMARY KEY      NOT NULL,  
  NAME TEXT              NOT NULL,  
  AGE INT                NOT NULL UNIQUE,  
  ADDRESS CHAR(50) ,  
  SALARY REAL           DEFAULT 50000.00  
);
```

Result:

Field	Type	Null	Key	Default
ID	int(11)	NO	PRI	
NAME	text	NO		
AGE	int(11)	NO	UNI	
ADDRESS	char(50)	YES		NULL
SALARY	double	YES		50000

Figure 11.2: Unique Constraint

11.2.3 Primary Key Constraint

1. The PRIMARY KEY constraint uniquely identifies each record in a database table.
2. There can be more UNIQUE columns, but only one primary key in a table. Primary keys are important when designing the database tables. Primary keys are unique ids.
3. A table can have only one primary key, which may consist of single or multiple fields. When multiple fields are used as a primary key, they are called a composite key.
4. If a table has a primary key defined on any field(s), then you cannot have two records having the same value of that field(s).

Example:

```
CREATE TABLE COMPANY(  
  ID INT PRIMARY KEY      NOT NULL,  
  NAME TEXT              NOT NULL,  
  AGE INT                NOT NULL,  
  ADDRESS CHAR(50) ,  
  SALARY REAL  
);
```

Result:

Field	Type	Null	Key	Default
ID	int(11)	NO	PRI	
NAME	text	NO		
AGE	int(11)	NO		
ADDRESS	char(50)	YES		NULL
SALARY	double	YES		NULL

Figure 11.3: Primary Key Constraint

11.2.4 FOREIGN KEY Constraint

1. A foreign key constraint specifies that the values in a column (or a group of columns) must match the values appearing in some row of another table.
2. They are called foreign keys because the constraints are foreign; that is, outside the table.
3. Foreign keys are sometimes called a referencing key.

Example:

Creates a table called COMPANY.

```
CREATE TABLE COMPANY(  
  ID INT PRIMARY KEY      NOT NULL,  
  NAME          TEXT      NOT NULL,  
  AGE           INT       NOT NULL,  
  ADDRESS       CHAR(50) ,  
  SALARY        REAL  
);
```

Creates a table called DEPARTMENT.

```
CREATE TABLE DEPARTMENT(  
  ID INT PRIMARY KEY      NOT NULL,  
  DEPT CHAR(50) NOT NULL,  
  EMP_ID INT      references COMPANY(ID)  
);
```

Result:

Field	Type	Null	Key	Default
ID	int(11)	NO	PRI	
DEPT	char(50)	NO		
EMP_ID	int(11)	YES		

Figure 11.4: Foreign Key Constraint

The column EMP_ID is the foreign key and references the ID field of the table COMPANY.

11.2.5 CHECK Constraint

1. The CHECK constraint ensures that all values in a column satisfy certain conditions.
2. If the condition evaluates to false, the record violates the constraint and isn't entered into the table.

Example:


```
CREATE TABLE COMPANY(
ID INT PRIMARY KEY      NOT NULL,
NAME                    TEXT    NOT NULL,
AGE                    INT      NOT NULL,
ADDRESS                CHAR(50) ,
SALARY                REAL      CHECK(SALARY > 0)
);
```

Result:

Field	Type	Null	Key	Default
ID	int(11)	☐ NO	PK	
NAME	text	☐ NO		
AGE	int(11)	☐ NO		
ADDRESS	char(50)	☐ YES		
SALARY	double	☐ YES		

Figure 11.5: Check Constraint

11.2.6 EXCLUSION Constraint

1. The EXCLUDE constraint ensures that if any two rows are compared on the specified column(s) or expression(s) using the specified operator(s), not all of these comparisons will return TRUE.
2. At least one of these operator comparisons will return false or null.

Example:

```
CREATE TABLE COMPANY(
ID INT PRIMARY KEY      NOT NULL,
NAME                    TEXT ,
AGE                    INT ,
ADDRESS                CHAR(50) ,
SALARY                REAL,
EXCLUDE USING gist
(NAME WITH =,
AGE WITH =)
);
```

Following data is inserted in the company table:-

```
INSERT INTO COMPANY10 VALUES(1, 'A', 20, 'abc', 20000.00 );
INSERT INTO COMPANY10 VALUES(2, 'B', 18, 'xyz', 25000.00 );
```

Result:

Data Output						Explain	Messages	History
	id	name	age	address		salary		
	integer	text	integer	character(50)		real		
1	1	A	20	abc		20000		
2	2	B	18	xyz		25000		

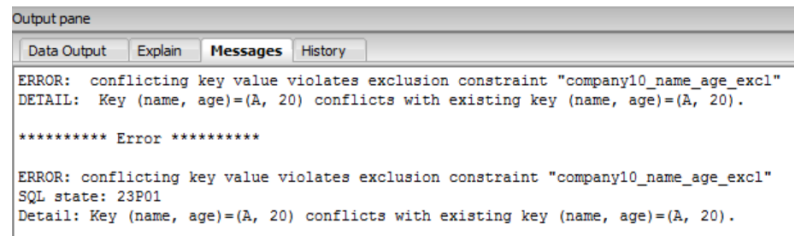
Figure 11.6: Exclusion Constraint (Before Inserting Excluded Data)

On adding the third row as:

```
INSERT INTO COMPANY10 VALUES(3, 'A', 20, 'pqr', 30000.00 );
```

The output is shown in Figure 11.7. An error is generated because the person with ID = 3 has same name and age as of person with ID = 1. Entries with same name and age cannot be entered into the table as they are excluded by using Exclusion Constraint.

Result:



```

Output pane
Data Output Explain Messages History
ERROR: conflicting key value violates exclusion constraint "company10_name_age_excl"
DETAIL: Key (name, age)=(A, 20) conflicts with existing key (name, age)=(A, 20).

***** Error *****

ERROR: conflicting key value violates exclusion constraint "company10_name_age_excl"
SQL state: 23P01
Detail: Key (name, age)=(A, 20) conflicts with existing key (name, age)=(A, 20).

```

Figure 11.7: Exclusion Constraint (After Inserting Excluded Data)

11.2.7 Dropping Constraints

1. To remove a constraint you need to know its name.
2. If the name is known, it's easy to drop.
3. Else you need to find out the system generated name.

Syntax:

```
ALTER TABLE table_name DROP CONSTRAINT some_name;
```

Example:

```
ALTER TABLE company MODIFY ID INT NOT NULL;
ALTER TABLE company DROP PRIMARY KEY;
```

The above query is applied on table company Figure 11.1 and primary key of ID is dropped. Output is shown in Figure 11.8.

Result:

Field	Type	Null	Key	Default
id	int(11)	NO		
NAME	text	NO		
AGE	int(11)	NO		
ADDRESS	char(50)	YES		NULL
SALARY	double	YES		NULL

Figure 11.8: Drop Constraint